

Lesson 9 — Automated Testing

Node.js/Express

Game Plan

- Warm-Up
- Why Automated Testing?
- Testing Concepts
- Jest Basics
- Supertest for API Testing
- Writing Good Tests
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. How have you been testing your app so far?
2. What's the most annoying thing about manual testing with Postman?

Testing So Far

You've been doing:

- **Manual testing** with Postman — you send requests, check responses by eye
- **Automated tests** provided by the course — they run against your code

The problem with manual testing:

- Slow and error-prone
- Easy to forget to test edge cases
- No way to know if a change broke something

Why Write Automated Tests?

It's a job requirement. Many teams won't merge code without tests.

Benefits:

- **Regression protection** — catch bugs introduced by changes
- **Documentation** — tests describe what your code should do
- **Confidence** — refactor without fear

In many shops: no test, no merge.

Types of Tests

Unit tests — test one function in isolation

Integration tests — test how multiple pieces work together

End-to-end tests — test the full stack (browser to database)

For this lesson, you'll write:

- Unit tests (individual functions)
- Integration tests for your Express API (using supertest)

Jest Basics

```
// math.test.js
const { add } = require("./math");

describe("add function", () => {
  it("should return 5 when adding 2 and 3", () => {
    expect(add(2, 3)).toBe(5);
  });

  it("should return 0 when adding negatives", () => {
    expect(add(-1, 1)).toBe(0);
  });
});
```

Run with: `npx jest` or `npm test`

Jest: Common Matchers

```
expect(value).toBe(5) // strict equality (primitives)
expect(value).toEqual({ a: 1 }) // deep equality (objects)
expect(value).toBeTruthy()
expect(value).toBeNull()
expect(value).toBeDefined()
expect(value).not.toBe(5) // negate any matcher
expect(array).toContain("item")
expect(value).toMatchObject({ name: "Alex" }) // partial match
```

Testing Async Code

Most of your code is async — use `async/await` in tests:

```
it("should find a user by email", async () => {  
  const user = await findUserByEmail("alex@test.com");  
  expect(user).not.toBeNull();  
  expect(user.name).toBe("Alex");  
});
```

If you forget `await`, the test may pass even when it should fail.

Supertest: Testing Your API

Supertest sends real HTTP requests to your Express app without needing a running server:

```
const request = require("supertest");
const { app } = require("../app");

it("should return 200 for GET /api/tasks", async () => {
  const res = await request(app)
    .get("/api/tasks")
    .set("Cookie", "token=valid_jwt_here");

  expect(res.status).toBe(200);
  expect(res.body).toHaveProperty("tasks");
});
```

Supertest: POST Example

```
it("should create a task", async () => {  
  const res = await request(app)  
    .post("/api/tasks")  
    .set("Cookie", `token=${validToken}`)  
    .set("X-CSRF-Token", csrfToken)  
    .send({ title: "Buy groceries" });  
  
  expect(res.status).toBe(201);  
  expect(res.body.title).toBe("Buy groceries");  
});
```

What Makes a Good Test?

Test one thing per test case.

```
// ❌ Too many assertions
it("should handle user creation", async () => {
  expect(res.status).toBe(201);
  expect(res.body.name).toBeDefined();
  expect(res.body.email).toBe("alex@test.com");
  expect(res.body.password).toBeUndefined();
});

// ✅ One assertion per test
it("should return 201 status on user creation", async () => {
  expect(res.status).toBe(201);
});
```

Test Edge Cases

Don't just test the happy path:

- What if the input is invalid?
- What if the record doesn't exist?
- What if the user isn't authenticated?

```
it("should return 400 for missing title", async () => {  
  const res = await request(app)  
    .post("/api/tasks")  
    .send({}); // no title  
  
  expect(res.status).toBe(400);  
});
```

Code Coverage

Jest can tell you how much of your code is tested:

```
npx jest --coverage
```

Output shows lines, branches, and functions that were never executed during tests.

Aim for high coverage — untested code paths are likely hiding bugs.

We Do — Write a Test Together

Let's write a test for this function:

```
function isValidEmail(email) {  
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);  
}
```

What test cases should we write?

- Valid email?
- Missing @ ?
- Empty string?

You Do (5 min)

Write two test cases for a `GET /api/tasks/:id` endpoint:

1. When the task exists — what should the response look like?
2. When the task doesn't exist — what status code do you expect?

Write just the `expect()` assertions first, then add the supertest setup.

TDD in This Course

The assignment provides tests **ahead of time**.

Your job:

1. Read the failing tests to understand what's expected
2. Write code that makes them pass

Some tests include **mocks** — fake versions of your functions that return wrong results. Your tests should catch those failures.

If your test passes on the mock, it's not a good enough test.

Assignment Preview

You'll write tests for your Express app:

1. Unit tests for helper functions (password hashing, validation)
2. Integration tests using supertest for:
 - User registration and login
 - Task CRUD operations
 - Auth protection (unauthenticated requests should return 401)
3. Edge cases: invalid inputs, missing records, wrong user

Run with: `npm run tdd assignment9`

Wrap-Up

In chat:

1. What's the difference between a unit test and an integration test?
2. Why should you test edge cases, not just the happy path?
3. What does `expect.assertions(n)` do and when would you use it?

Confidence Check

On a scale of 1–5:

How confident do you feel writing automated tests this week?

Resources

- <https://jestjs.io/docs/expect> (all Jest matchers)
- <https://www.npmjs.com/package/supertest>
- <https://www.functionize.com/automated-testing> (testing concepts overview)
- Ask questions in Slack

Closing

This week:

Automated testing — Jest, supertest, and TDD.

Next week:

Deployment — get your app onto the real Internet using Neon and Render.

See you then!